NATS-Bench-MCU — Summary of Changes for the Camera-Ready Version

This document summarizes the changes made to address the reviewers' and Area Chair's comments, as requested for the camera-ready submission. Reviewers ZdDi and UjGz raised one shared **major** concern (an internal inconsistency in the reported failure sizes) plus several clarity/consistency fixes; the Area Chair asked us to address ZdDi and UjGz specifically.

---

1. Failure taxonomy and precheck threshold (ZdDi & UjGz — major)

Both reviewers observed that the reported failure sizes are internally inconsistent: the "precheck" failure class had a minimum size of **360 KB** (ZdDi) even though the precheck threshold is 800 KB, and there appeared to be no label for architectures in the **~746–800 KB** band (UjGz). We traced this to its root and corrected it.

1.1 Root cause

The deployment flash cost is, to within measurement precision, an exact affine function of the model size:

```
flash_used = 277.6 KB (constant firmware:  Zephyr base + TFLite-Micro + app) + INT8 model
flatbuffer
```

The firmware footprint is **277.583 KB (284 245 bytes)**, constant across all 14 281 successful builds, because the model flatbuffer is embedded verbatim as a C array on top of a fixed firmware image. On the 1024 KB application-core flash this gives a **true deployable ceiling of 746 KB**.

Historically we have set a 80% of a conservatively declared 1000 KB flash = **800 KB** as a threshold. While this worked out for many of our previous experiments, the zephyr firmware used for this benchmark, was slightly bigger than the firmwares we used before. So our actual budget for an architecture was 746 KB, ~54 KB less than the threshold. Two consequences followed, one per reviewer:

1.2 The 746–800 KB band (UjGz)

115 architectures with INT8 flatbuffers of **747–791 KB** passed the 800 KB precheck but were then rejected by the linker's flash-overflow check (`flash_failed`) before ever reaching the device. In the released export these `flash_failed` outcomes were folded into the `precheck` class, so the band was not visibly labelled. They are, in every case, **flash-infeasible** (model + 277.6 KB firmware > 1024 KB) and produced **no** device measurements.

Fix. We tightened the precheck to reserve the exact firmware footprint, so it now gates on the true 746 KB budget and rejects these 115 architectures *up front at the precheck stage*, matching the linker. We verified on the full space that the corrected precheck (i) admits all 14 281 deployable architectures (largest = 744.3 KB) and (ii) rejects all 1 344 infeasible ones (smallest = 747.0 KB) — a clean separation with no false rejections and no leaks. All flash-infeasible rejections are now

reported under a single outcome label, and the released raw error logs for the 115 band architectures were regenerated to the precheck format so the artifacts agree with the exported tables.

Label rename. With only two outcome labels remaining, we renamed the failure label from the internal pipeline-*stage* name `precheck` to the *outcome* name `infeasible`, so the released labels (`success` / `infeasible`) describe outcomes symmetrically rather than mixing an outcome with a stage name. The word "precheck" is retained only where it names the pipeline stage/mechanism (the check is still performed at, and cheaply detected by, the flash-feasibility precheck). This affects the paper text, the `stage` column of `failures.csv` (value `precheck` → `infeasible`), and the export code; the MIMaaS server and the raw per-architecture artifacts keep the server's internal stage name `precheck`, from which the export derives the `infeasible` outcome label.

1.3 The 360 KB anomaly (ZdDi)

The two sub-threshold "failures" (359.8 KB and 362.7 KB, architectures 7650 and 7651) were **not flash failures at all** — both are small and actually fully deployable. They were casualties of transient *server-infrastructure* bugs, not of the flash budget:

- **Arch 7650** was in fact **fully and correctly measured** on 2026-05-04, but its request status was left at `pending` by an orphaned-process/queuing bug in the measurement server. Because it looked unfinished, it was resubmitted; by then the server code was in an unstable editing window and the precheck spuriously rejected it. That later spurious rejection overwrote the record, so the architecture reached the database as a "360 KB precheck failure." but was not discovered until the camera-ready review.
- **Arch 7651** was built successfully on 2026-05-04 but the pipeline died mid-flash (a GPIO-timeout / orphaned-process bug), leaving no device measurements.

In short, the 360 KB minimum was an **artifact of two infrastructure failures being mislabelled as flash rejections**, not a real property of the precheck. (The exact transient code state at the time is not recoverable, but the nonsensical rejection message makes the mislabelling unambiguous.)

Fix. We recovered arch 7650's genuine measurement from the server store and remeasured arch 7651 on the nRF5340-DK with the corrected pipeline. Both are now normal completed records; their firmware overhead (rom − int8) is 277.6 KB, identical to every other build, confirming consistency. The `infeasible` failure class therefore no longer contains any sub-746 KB member — the anomaly is resolved.

We will update the Zenodo release with the corrected export.

1.4 Resolution and corrected numbers

Quantity	Original (submission)	Corrected (camera-ready)
Successful deployments	14 279	**14 281**
Rejected (flash-infeasible)	1 346	**1 344**
Failure taxonomy	binary (implicitly)	single `infeasible` class, all flash-infeasible
Precheck / deployable ceiling	800 KB (80 % heuristic)	**746 KB** (1024 − 277.6 KB firmware)
Min size in failure class	360 KB (anomaly)	**747.0 KB**
Failed INT8 size (mean ± sd)	934 ± 124 KB	**933.8 ± 122.8 KB**
Successful INT8 size (mean / median)	353 / 356 KB	**352.7 / 356.0 KB**
ROM usage range	363 – 1022 KB	363.0 – 1021.9 KB

2. Quantization and the accuracy ranking (ZdDi)

> *"Real measurements (latency and energy) are paired with the FP32 accuracy and not the actual INT8 accuracy ... The authors argue that the ranking is not affected by the quantization, but do not back that claim up with data."*

We add an experiment (`int8_rank_correlation.py` → `int8_rank_analysis.py`; the paper's Figure 5) that tests this claim directly. **What it tests:** whether INT8 post-training quantization — the exact full-integer PTQ pipeline used to produce the deployed flatbuffers — preserves the *accuracy ranking* of architectures, which is the property NAS actually relies on.

Protocol. We use **CIFAR-10** throughout, because the models actually compiled, flashed, and measured on the nRF5340 are the CIFAR-10 variants (10-class head, 32×32×3 input — verifiable in the released flatbuffers); the CIFAR-10 accuracy column is therefore the one paired one-to-one with each deployed INT8 model, so training the ranking experiment on CIFAR-10 keeps the architecture identical to what was deployed. We draw a **stratified sample of 100 deployable architectures** spanning the full CIFAR-10 accuracy range, train each from scratch on CIFAR-10 (12-epoch schedule, SGD + cosine decay, standard augmentation), and evaluate two accuracies on the *same* test set: the trained model in FP32 (Keras) and the same weights after full-integer INT8 PTQ (calibrated on real CIFAR-10, evaluated through the TFLite interpreter with uint8 I/O — identical to the deployment path). We then measure rank agreement two ways:

- **(a) The quantization step itself** — our FP32 → our INT8 (same weights, same test set, only PTQ differs). This is the *direct* test of the reviewer's point.
- **(b) The full tabulated-to-deployed chain** — the tabulated NATS-Bench FP32 accuracy (what we actually pair with the hardware costs) → our on-device INT8 accuracy.

Result (a): quantization does not affect the ranking. Across the 100 architectures INT8 PTQ changes test accuracy by only **0.27 pp on average** (median 0.15 pp, max 1.90 pp), and the rank correlation is **near-perfect**:

Comparison (n = 100)	Spearman ρ	Kendall τ	Top-10 overlap	Top-25 overlap	Mean $\lvert\Delta\text{acc}\rvert$
(a) FP32 → INT8 (quantization only)	**0.999**	**0.982**	9 / 10	25 / 25	**0.27 pp**
(b) tabulated NATS FP32 → INT8	0.903	0.736	4 / 10	21 / 25	—

There were **zero genuine quantization craters** (no architecture with a non-trivial FP32 accuracy lost more than 10 pp under INT8). Two architectures sit at CIFAR-10 chance (≈10 %) in *both* FP32 and INT8 — they are degenerate all-`none` cells that NATS-Bench itself scores at 10 %, i.e. dead *before* quantization, not harmed by it; excluding them leaves the correlation unchanged ($\rho = 0.999$).

Robustness of result (a). Three checks confirm the near-perfect (a) is not a sample-size or sampling artifact:

- *Sample size.* A 10 000-fold bootstrap over the 100 architectures gives a tight 95 % CI of $\rho \in$ **[0.997, 0.999]** and $\tau \in$ **[0.97, 0.99]** — n = 100 is statistically ample here. (The load-bearing

evidence is in any case the per-architecture effect — mean 0.27 pp, max 1.90 pp, no craters — which a correlation only summarizes; a perturbation that small cannot reorder much.)

- *Not a full-range artifact.* The architectures are drawn **stratified** — one at random from each of 100 equal-count bins of the deployable set sorted by CIFAR-10 accuracy — so they span the whole range rather than over-weighting the dense middle. Restricting the correlation to the high-accuracy band where NAS actually operates keeps it near-perfect: for FP32 ≥ 80 % (n = 36) $\rho = 0.994$ / $\tau = 0.956$, and *within the 25 best architectures* $\rho = 0.995$ / $\tau = 0.968$.
- *Decision regret.* Selecting the single best architecture by FP32 accuracy yields exactly the best architecture under INT8 (top-1, top-3, and top-5 selection regret all **0.00 pp**) — the property NAS ultimately cares about.

Scope. These architectures are trained on a compact 12-epoch schedule (to keep 100 from-scratch trainings tractable), so strictly we demonstrate rank-preservation under quantization *for these models*; that the effect is ≈ 0.27 pp with zero craters, together with the well-established robustness of full-integer PTQ for small convolutional models, makes the same conclusion very likely to hold for fully-trained weights, but we state the training budget explicitly rather than imply a 200-epoch result.

3. Demonstration of benchmark utility for NAS (ZdDi)

> *"The paper is framed as a NAS benchmark ... but its utility as such is stated but never demonstrated. No NAS algorithm is ever evaluated."*

We add a demonstration (`nas_baselines.py`; the paper's Figure 6) running the three standard NATS-Bench search algorithms — Random Search (RS), Regularized Evolution (REA), Local Search (LS) — as pure table lookups over the corrected benchmark, in a hardware-aware setting: **maximize CIFAR-10 test accuracy subject to an on-device energy budget** (here the median deployable energy, 36.8 mJ). Under this budget only **7,141 / 15,625 (45.7%)** architectures are feasible and the oracle accuracy is 93.25 %. Each algorithm is run for 100 independent seeds of 300 evaluations.

4. Practical relevance of the measured latency and energy (ZdDi)

We add a short discussion putting the measured operating point in context. Across the 14 281 deployable architectures the measured cost spans a **median inference latency of 3.18 s** (range 0.33–6.69 s) at a **mean power of 11.7 mW** (range 10.3–12.5 mW), for a **median energy of 36.8 mJ per inference** (range 3.8–79.4 mJ).

5. Minor corrections (UjGz)

- **ROM range consistency (line 214 vs Table 1).** The text's "363 to 1,022 KB" is the range of *used* ROM; Table 1's upper value 1,024 is the *physical* flash size. We reconcile these: max ROM actually used is 1021.9 KB (≈ 1022); the 1024 KB figure is labelled explicitly as the physical flash capacity.

- **Abstract metric wording (line 14).** "current draw" → **"mean power"**, to match the paper's measurement section, which deliberately reports mean power (current × 3.3 V nominal).
- **Figure 1 (pipeline) stages.** The caption previously referred to "stages one and two" / "stages three through six" with no corresponding numbers in the diagram; we removed that unlabelled numbering. The full pipeline (model construction → INT8 conversion → flash-feasibility precheck → firmware build → flash → on-device measurement) is enumerated in Section 3.3.
- **"~25 %" anchor (lines 207–208).** The failed mean INT8 size exceeds the **deployable maximum of 744 KB** by ≈ 25 % (933.8 / 744.3 ≈ 1.255); the anchor (744 KB, not the 800 KB threshold) is now stated explicitly.

---

6. Data and code availability

The corrected dataset and figures are regenerated deterministically by the released code (`recover_corrections.py` → `export_plot_data.py` → `plot_from_export.py`); the correction is carried by a small `corrections/` overlay applied on top of the original artifacts, documented in `corrections/README.md`. The corrected precheck is in the released `mimaas-server` code. The two new experiments are self-contained and reproducible: the NAS demonstration (§3) is `nas_baselines.py` → the NAS figure (Figure 6; pure table lookups over the corrected CSVs, seconds to run), and the quantization-ranking study (§2) is `int8_rank_correlation.py` (the GPU training/evaluation run) → `int8_rank_analysis.py` → the quantization figure (Figure 5), with per-architecture results in `export_corrected/int8_rank_correlation.csv` and the summary statistics in `export_corrected/int8_rank_summary.csv`. A new Zenodo version hosts the corrected export tables and overlay; the original version remains available and is referenced for provenance.